Pointer vs Unsigned integer

→ All Addresses are whole numbers (+ve).

- Unsigned int V; // is an Integer variable, not a pointer.

```
main(){ unsigned int x=5000, y=6000;
        x = &y;
Pf("x = %u  &y = %u", x, &y);
(*x) = 7000;
}
```

x: 5000 → 8620 (8600)

y: 6000 (8620)

syntax Error as x is NOT a pointer, hence De-reference NOT allowed

```
main(){
unsigned int y = 6000;
unsigned int *x = &y;
Pf("y = %d \n", y); // 6000
*x = 7000; // y = 7000
Pf("y = %d \n", y); }
```

Declarator, NOT operator

reference operator

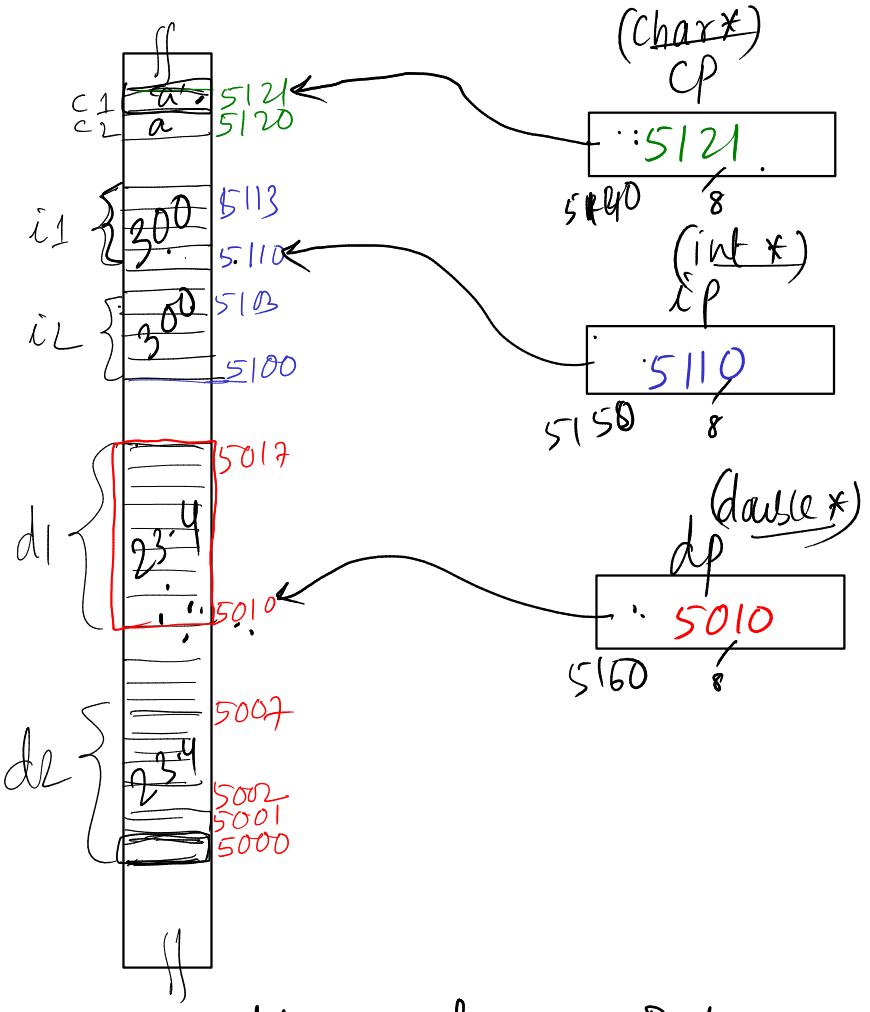
x: 5500 (5520) → y: 6000 → 7000 (5500)

De-reference operator

x is having reference of y.

hence De-ref of x is y.

```
unsigned int *x;
x = &y;
```



- All Pointers hold Address of a Byte.
  - ↳ Base Address

- All pointers are of same size.

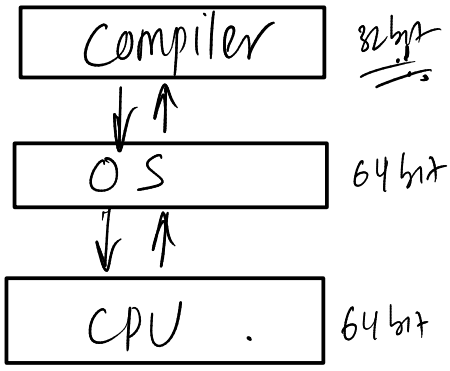
```
sizeof(cp); //8
sizeof(ip); //8
sizeof(dp); //8

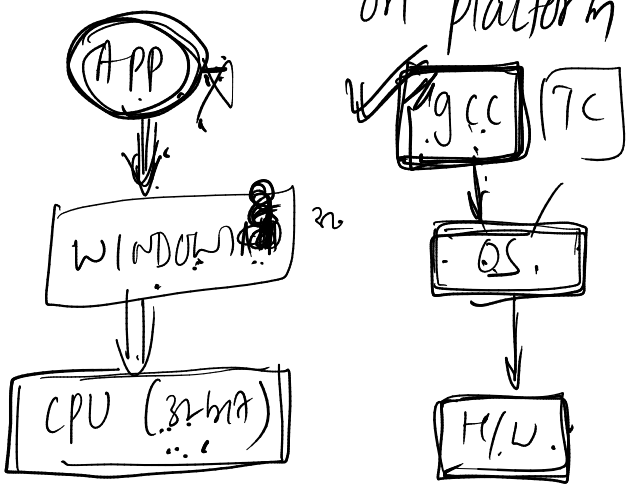
sizeof(*cp); //1
sizeof(*ip); //4
(*dp); //8
```

Size of pointer

→ Based on platform, may be 2/4/8 bytes.

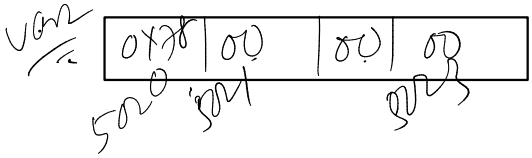
- 1 byte pointer not used
- 2 bytes pointer are used in tiny/small mem models
- Normally pointers are 4 or 8 bytes depending on platform

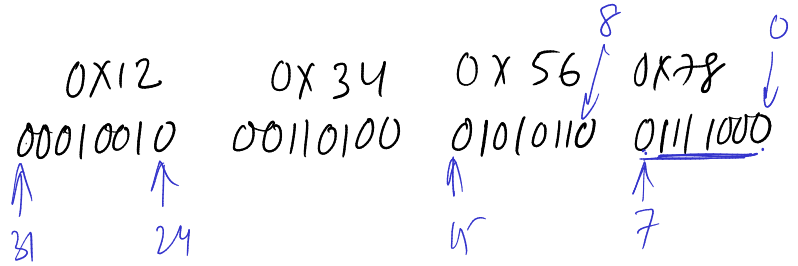


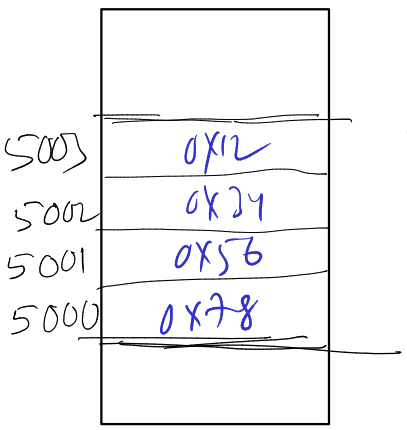


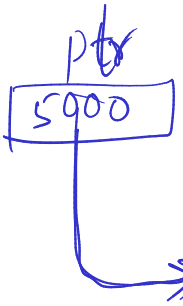
---

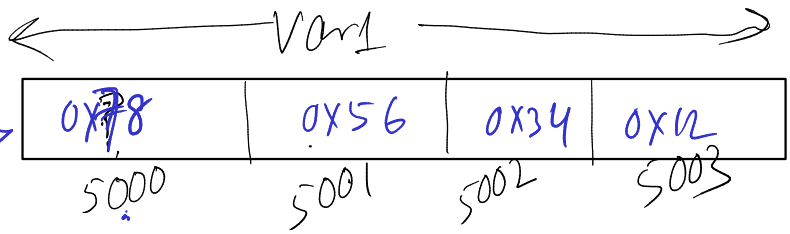
var1 => 0x12345678











char ch = 0x78;

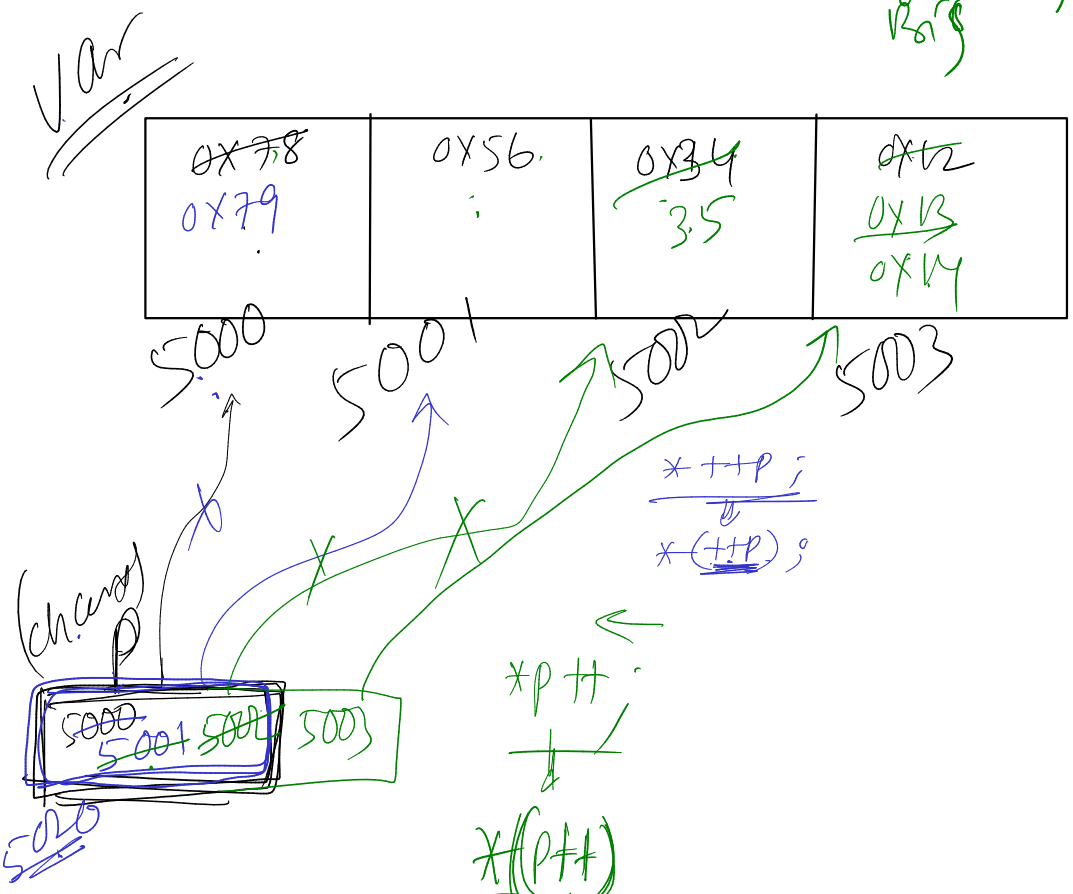
unsigned int v; v = ch.
= 0x00000078

CPU - Byte ordering

- Little Endian
  → Lower byte ~~in lower location~~ first

Intel

- Big Endian
  → Higher Byte first



0X12 3456 78

++(*p)

*++p ;
*(++p);

*p++ ;

*((p++))

printf("%x", ++*p); // 79

%x", *++p); // 56

*p++); // 56

++*p) // 35

*(p++)) // 35

(*p)++ // 12

++*p // 14

("%x", var);